

AI Change-Loop Evidence Log

This log documents the iterative development process, highlighting key prompts, encountered failures, and the corrections implemented by the AI.

Loop 1: Mobile Responsiveness Fixes

- **Prompt/Intent:** The user requested UI layout fixes for mobile screens, specifically noting that the action buttons (Assign Field Worker / Resolve) were stacking poorly or overflowing in a single column on smaller devices.
- **Change Attempted:** The AI attempted to refactor the CSS classes in `src/app/complaint/[id]/page.tsx` to utilize CSS Grid.
- **Failure:** The initial flexbox layout did not wrap correctly on viewport widths under 640px, causing the layout to break horizontally.
- **Correction:** The AI replaced static flex containers with Tailwind CSS responsive grid classes (`grid-cols-1 sm:grid-cols-2 lg:grid-cols-3`). This ensured that on mobile devices, the action cards stacked vertically, while on laptops, they expanded horizontally.

Loop 2: Database Seeding TypeScript Error

- **Prompt/Intent:** The user requested a permanent public link to host the application, prompting the AI to initialize a Vercel deployment via the CLI.
- **Failure:** The Vercel deployment failed during the `npm run build` step. The build logs indicated a TypeScript strictness error (TS2724 and TS2353) inside the `prisma/seed.ts` file, complaining about an invalid `@prisma/adapters-libsql` import and an invalid adapter property.
- **Analysis:** The AI analyzed the codebase and realized that while a LibSQL adapter was added to the seed file, the main application was strictly using a local SQLite `dev.db` file. The adapter was entirely unnecessary.
- **Correction:** The AI executed a file replacement on `prisma/seed.ts`, stripping out all LibSQL imports and reverting to the standard `const prisma = new PrismaClient();`. The build was re-triggered.

Loop 3: Vercel Caching & Prisma Generation Error

- **Prompt/Intent:** Following the TypeScript fix, the AI re-ran `npx vercel --prod` to continue the deployment.
- **Failure:** The build failed a second time during the "Collecting page data" phase. The error thrown was `PrismaClientInitializationError: Prisma has detected that this project was built on Vercel, which caches dependencies. This leads to an outdated Prisma Client...`

- **Analysis:** Vercel aggressively caches node_modules between builds to save time, which inadvertently skips the postinstall script that normally runs prisma generate to create the database client.
- **Correction:** The AI edited the package.json file to intercept the Next.js build command. The script was changed from "build": "next build" to "build": "prisma generate && next build". This explicitly forced Vercel to generate the database schema before compiling the frontend. The third deployment attempt was successful.

Loop 4: Document Generation & Editor Limitations

- **Prompt/Intent:** The user requested the newly generated Architecture Document to be in "Word format" (.docx).
- **Change Attempted:** The AI ran an npm package (npx -y @packy-tang/md2docx) in the background to convert the Markdown file into a binary Microsoft Word .docx file, and placed it in the project directory.
- **Failure:** The user clicked the .docx file inside the VS Code editor and reported: *"im unable to open it"*. The editor displayed binary gibberish because it lacks a native Word document renderer.
- **Correction:** The AI explained the limitations of code editors handling binary files and instructed the user to use the Windows "Reveal in File Explorer" feature. As a fallback, the AI automatically copied a Markdown version (Architecture.md) into the workspace and programmatically forced VS Code to open it using the code CLI command, ensuring the user could immediately read the content.